



Univerza v Mariboru

Fakulteta za elektrotehniko,
računalništvo in informatiko

ARHITEKTURNE ODLOČITVE

TraceBit

Luka Majstorović & Izidor Robnik

Kazalo

ARHITEKTURNE ODLOČITVE	1
TraceBit	1
1. Razdeljen (modularen) microservice-design	1
1.1. Frontend (Next.js + App Router + TypeScript + Tailwind CSS)	1
1.2. Backend (FastAPI + Python)	2
2. Shranjevanje in upravljanje podatkov	4
3. Zbiranje in prenos fingerprint podatkov	5
4. Odločitev glede detekcije anomalij / uporabe strojnega učenja	5
5. Avtentikacija, vloge in varnostni mehanizmi	6

1. Razdeljen (modularen) microservice-design

Arhitekturo smo zasnovali kot **ločeni aplikaciji** za frontend in backend. Ta modularen pristop je bil ključen za izboljšano **ločevanje odgovornosti**, **neodvisen razvoj** posameznih komponent ter enostavnejše testiranje in razhroščevanje. Ločitev sistema na dva ločena sklopa nam omogoča, da se spremembe v enem delu sistema (UI) ne odražajo neposredno na drugem (backend), kar povečuje **odpornost sistema** in omogoča lažji prihodnji razvoj ali zamenjavo enega od delov brez vpliva na celoto.

1.1. Frontend (Next.js + App Router + TypeScript + Tailwind CSS)

Za frontend smo izbrali **Next.js** (z App Routerjem) skupaj z **TypeScriptom** in **Tailwind CSS**. Odločitev smo sprejeli na podlagi več dejavnikov:

- **Enostavnost uporabe:** Ker naša prednost ni znanje na področju frontend razvoja, smo želeli uporabiti ogrodje, ki omogoča hitro izdelavo funkcionalnega in odzivnega uporabniškega vmesnika brez potrebe po kompleksnem razvoju.
- **Next.js** nudi dobro privzeto strukturo, podporo za SSR (server-side rendering), API route-e, ter močno skupnost in dokumentacijo, kar je pomembno za hitro učenje in odpravljanje težav.
- **App Router** (nov sistem usmerjanja v Next.js) nam je omogočil boljšo organizacijo strani in komponent, kar je bilo ključno za doseganje cilja modularne gradnje (user/admin sekcije).
- **Tailwind CSS** smo uporabili zaradi njegove hitrosti razvoja UI elementov, saj omogoča hiter styling brez pisanja CSS razredov od začetka, kar je še posebej pomembno pri našem timu z omejenim znanjem oblikovanja.

Skratka, za ta pristop smo se odločili zato, ker je bil za nas **najlažja, najhitrejša in najmanj tvegana rešitev**, ki hkrati omogoča skalabilnost in profesionalen rezultat.

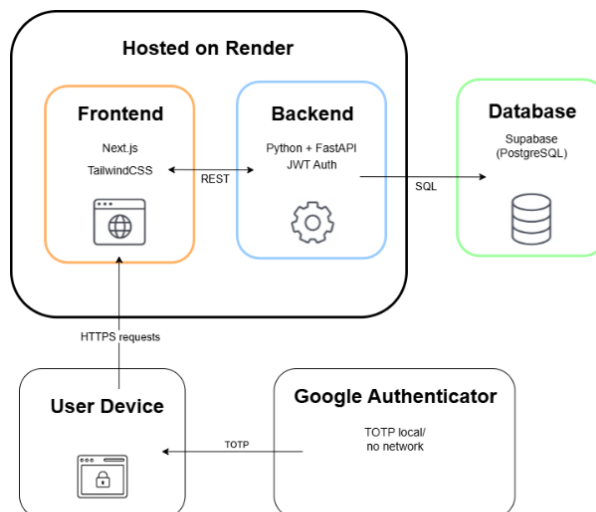


1.2. Backend (FastAPI + Python)

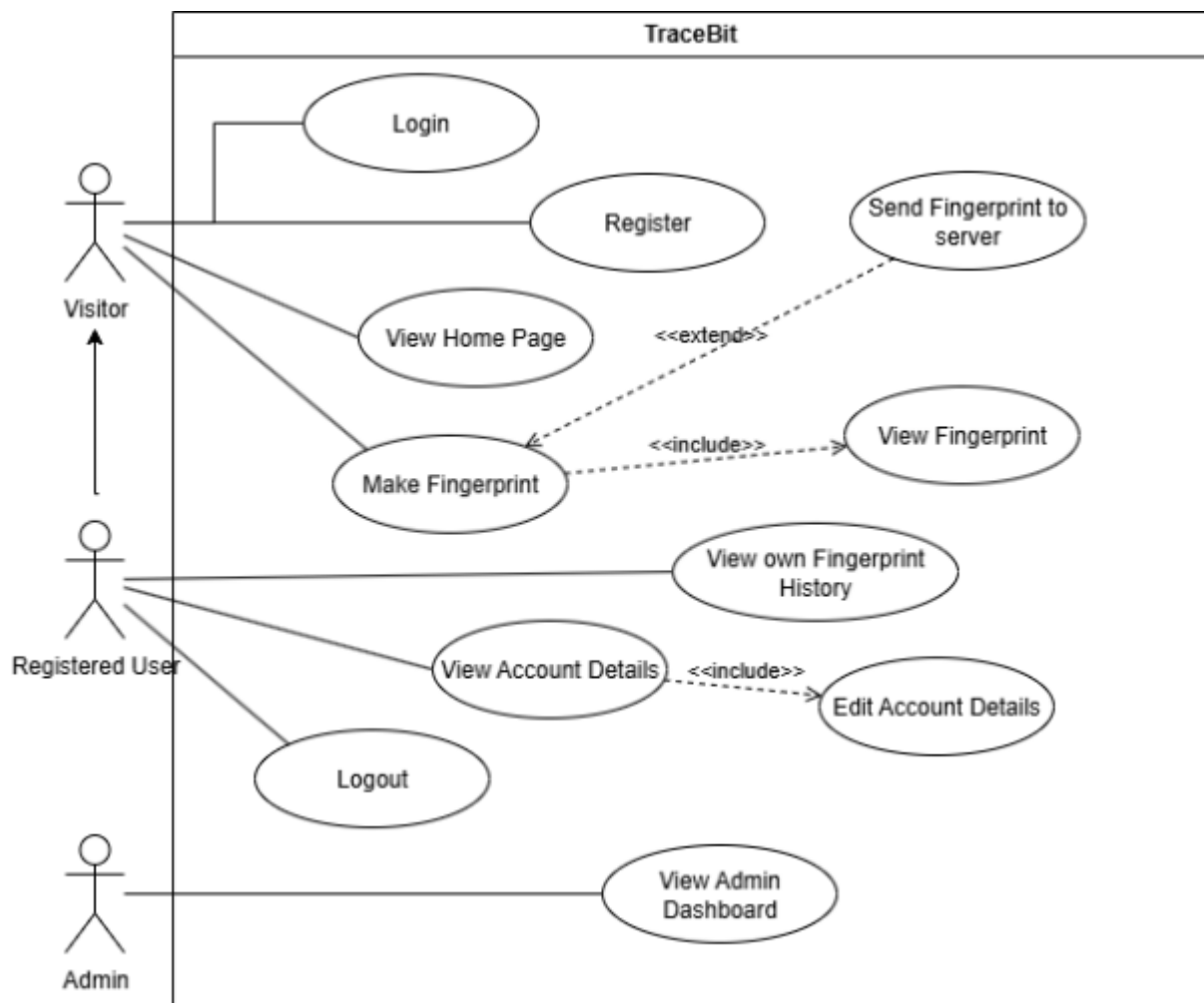
Za backend del sistema smo uporabili **FastAPI**, moderno ogrodje zgrajeno v Pythonu, saj nudi kombinacijo visoke zmogljivosti in enostavne implementacije. Glavni razlogi za izbiro so bili:

- **Hitro definiranje REST endpointov** s pomočjo OpenAPI specifikacije in avtomatsko generirane dokumentacije (Swagger UI).
- **Zelo dobra integracija s Python ekosistemom** (npr. uvajanje ML logike, povezava z bazo prek Supabase SDK).
- **JWT avtentikacija** je preprosto izvedljiva in se zlahka kombinira z RLS politiko v Supabase bazi.
- **Asinhronost** v FastAPI omogoča visoko zmogljivost pri sočasnem sprejemanju zahtevkov, kar je pomembno pri analizi številnih fingerprint zahtev.

Ker imamo več izkušenj z Python okoljem, nam je FastAPI predstavljal naravno izbiro, ki omogoča hitro implementacijo poslovne logike, brez večjih krivulj učenja.



Slika 1 Arhitektura sistema TraceBit z ločenim frontendom, backendom in bazo podatkov Supabase.



Slika 2 Uporabniški primeri (Use Cases)

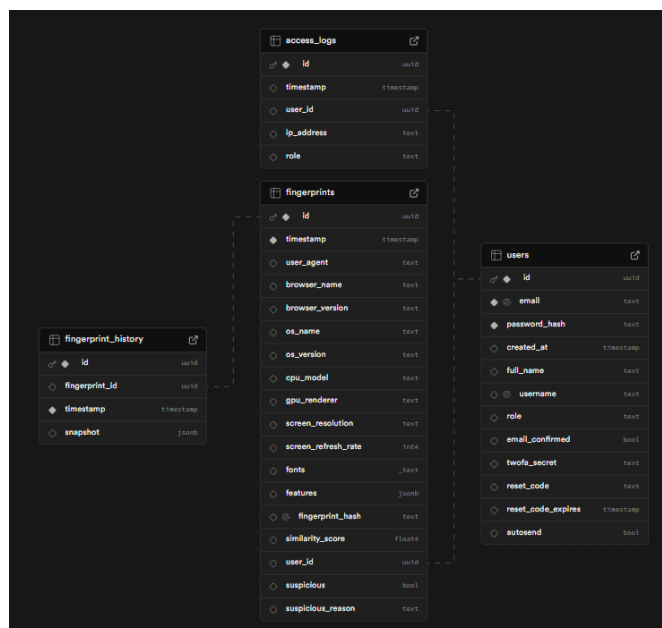
2. Shranjevanje in upravljanje podatkov

Za podatkovno shranjevanje smo uporabili **Supabase (PostgreSQL)**, saj omogoča hiter začetek, vgrajeno **avtentikacijo**, **RLS (Row Level Security)** ter brezplačen hosting za manjše projekte. Zaradi enostavne integracije s FastAPI in odličnega uporabniškega vmesnika je bila to za naš projekt optimalna izbira brez potrebe po dodatnem ORM sloju.

Struktura baze:

- **users:** vsebuje uporabnike z vlogami (user/admin), 2FA, potrjen email in povezavo do njihovih fingerprintov.
- **fingerprints:** hrani hash in tehnične podatke (userAgent, canvas, WebGL, OS, brskalnik, CPU, GPU itd.), vezane na user_id.
- **fingerprint_history:** hrani zgodovinske posnetke fingerprintov (snapshot) z datumom in povezavo na fingerprint_id.
- **access_logs:** beleži prijave admin uporabnikov z IP naslovom, časom in vlogo.
- **logs** (prek triggerjev): z uporabo triggerjev shranjuje dogodke, kot so prijave ali shranjevanje fingerprintov, brez dodatne obremenitve backend kode.
- **anomalies** (integrirano znotraj fingerprints): vsebuje oznako suspicious in razlog suspicious_reason za zaznane sumljive odtise.

Vse komunikacije z bazo potekajo prek **FastAPI**, kjer izvajamo JWT preverjanje, validacijo in dostop glede na vlogo uporabnika. S tem smo dosegli varno, pregledno in razširljivo podatkovno osnovo za sistem.



Slika 3 ER diagram

3. Zbiranje in prenos fingerprint podatkov

Na frontend strani smo v TypeScriptu implementirali funkcijo, ki zajema ključne podatke o uporabnikovi napravi in okolju. Med drugim zbiramo:

- navigator.userAgent (za pridobitev OS, brskalnika, CPU),
- lastnosti zaslona (ločljivost, refresh rate),
- jezik sistema, časovni pas, touch support,
- WebGL, canvas hash in druge identifikacijske značilnosti.

Zajeti podatki se pošljejo na backend prek POST zahteve na FastAPI endpoint /fingerprint. Na backendu se iz teh podatkov izračuna deterministični hash (SHA-256), ki služi kot identifikator naprave. Ta pristop omogoča poenostavljeno primerjanje fingerprintov in zaznavanje razlik med njimi.

4. Odločitev glede detekcije anomalij / uporabe strojnega učenja

Sprva smo načrtovali uporabo **modela strojnega učenja (ML)** za zaznavanje anomalnih fingerprintov. Vendar pa smo se zaradi **premajhne količine podatkov** odločili, da idejo prilagodimo. Uporaba ML pristopa bi bila smiselna le z veliko raznolikih primerov, zajetih skozi daljše časovno obdobje, kar trenutno ni mogoče.

Namesto tega smo implementirali niz **ročno definiranih pravil in heuristik**, ki jih imenujemo "osnovna" in "napredna" logika detekcije (npr. sprememba OS ali CPU, ponavljajoče vrednosti, redke kombinacije).

- Python ML knjižnice kot so scikit-learn, TensorFlow ali XGBoost **niso bile uporabljene**, saj bi za učinkovito učenje modela potrebovali **tisoče realnih fingerprintov**, skupaj z evalvacijo preko ROC krivulj, kros-validacije in ustrezni labeliranimi podatki. Kljub temu je arhitektura sistema pripravljena na kasnejšo vključitev ML modela, ko in če bodo pogoji ustrezni

5. Avtentikacija, vloge in varnostni mehanizmi

Dostop do API-jev in podatkov je zaščiteno s pomočjo **JWT (JSON Web Token)** mehanizma.

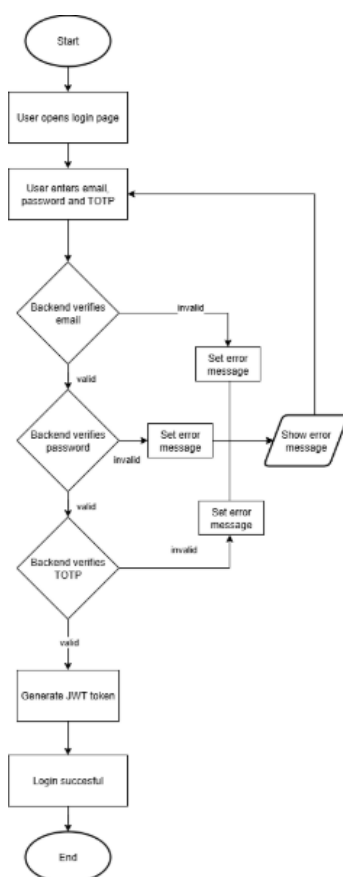
Uporabniki so razdeljeni na dve vloge:

- **"user"** ima dostop samo do svojih podatkov,
- **"admin"** ima dostop do vseh fingerprintov in statistike.

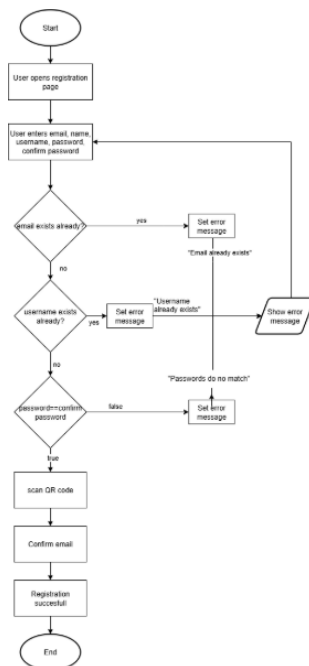
Pri registraciji uporabnika se izvaja **2FA**, vključena je tudi možnost ponastavitve gesla z uporabo potrditvene kode in časovno omejenega žetona. Gesla so varno **hashirana z bcrypt**, vse povezave pa potekajo prek HTTPS.

Poleg tega uporabljamo RLS (Row-Level Security) politike v Supabase, ki na ravni baze zagotavljajo, da vsak uporabnik vidi le svoje podatke.

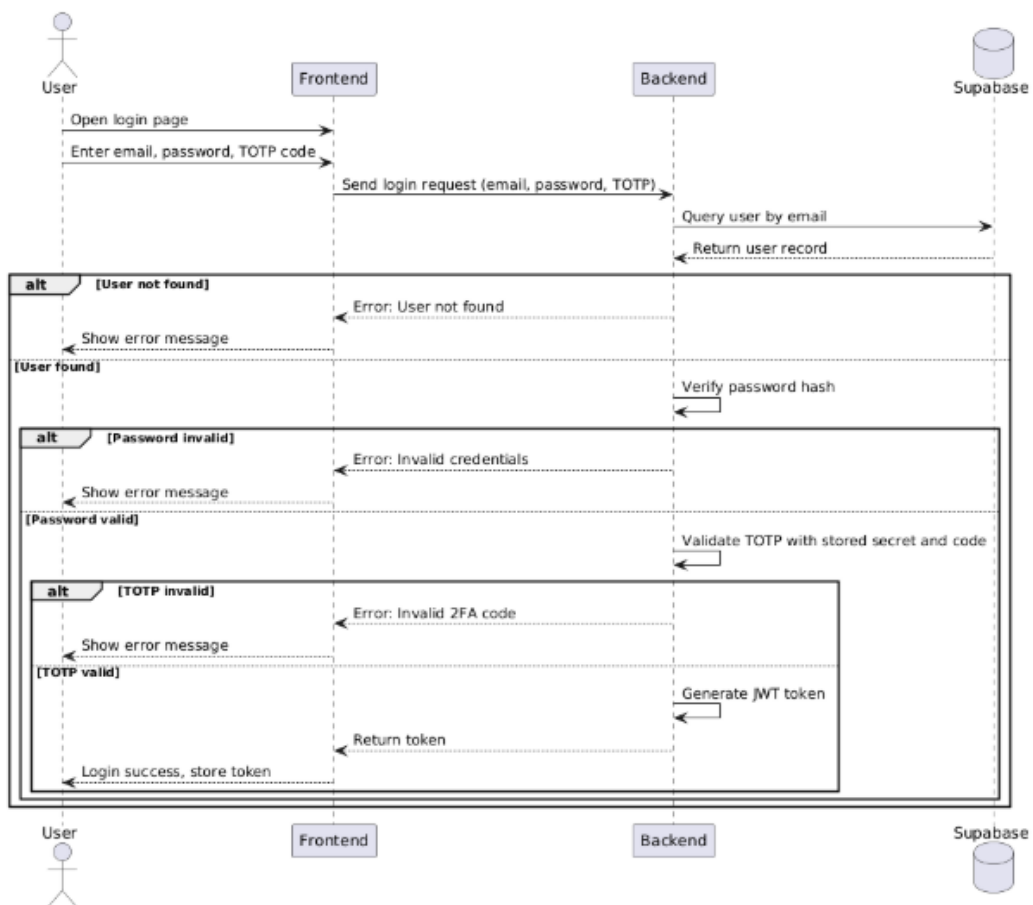
Na frontend in backend aplikacijah so dodatno nastavljeni CORS mehanizmi, ki preprečujejo neželene zahteve iz nepooblaščenih virov.



Slika 4 Prikazuje komunikacijo med uporabnikom, frontendom, backendom in Supabase med postopkom prijave.



Slika 5 Prikazuje logiko in validacijo pri registraciji, vključno s preverjanjem emaila, uporabniškega imena in gesla.



Slika 6 Logični prikaz preverjanja prijave (email, geslo, TOTP).